Blog Platform

Submitted by: NEHAL MEHTA

Introduction

The Blog Platform is a full-stack MERN (MongoDB, Express.js, React.js, Node.js) web application that allows users to create accounts, publish blog posts, and interact with content created by other users. The system focuses on secure authentication, clean user experience, and structured backend architecture.

The objective of this project was to design and implement a user-centric blogging platform that fulfills authentication, CRUD operations, responsive design principles, and secure data handling using modern web development technologies.

Objectives

The main objectives of this project were:

- To implement secure user authentication using JWT.
- To enable authenticated users to perform full CRUD operations on blog posts.
- To maintain clean project structure separating frontend and backend.
- To ensure secure password storage using hashing techniques.
- To create a responsive and intuitive user interface.
- To implement proper route protection and authorization logic.
- To follow RESTful API architecture principles.

Technologies Used

Frontend

- React.js
- React Hooks (useState, useEffect)
- react-router-dom
- Axios / Fetch API
- CSS for styling

Backend

- Node.js
- Express.js
- JSON Web Tokens (jsonwebtoken)
- bcrypt.js (Password hashing)
- CORS

Database

- MongoDB (Cloud-hosted compatible)
- Mongoose (Schema & data modeling)

Other Tools

- dotenv (Environment variables)
- Postman (API testing)
- Git & GitHub (Version control)

System Architecture

The project follows a **client-server architecture**:

Frontend (React)

- Handles UI rendering
- Manages authentication state
- Sends HTTP requests to backend APIs
- Stores JWT token in localStorage for persistence

Backend (Express)

- RESTful API design
- Handles authentication and authorization
- Interacts with MongoDB database
- Validates requests and protects private routes

Database (MongoDB)

- Stores users, blog posts, and comments
- Uses Mongoose schemas for structure and validation

Core Functionalities Implementation

User Management & Authentication

User Registration

- Users register with unique username/email and password.
- Passwords are hashed using bcrypt before storing in the database.
- Duplicate user validation implemented.

User Login

- Credentials verified against hashed passwords.
- Upon successful login, a JWT token is generated and sent to the client.
- Token is stored in localStorage for session persistence.

Authentication Persistence

- Login state is maintained across page refreshes using JWT stored in localStorage.
- Protected routes check token validity before granting access.

Route Protection

- Backend middleware verifies JWT.
- Frontend PrivateRoute component restricts access to authenticated users.
- Only post authors can edit or delete their posts.

Blog Post Management (CRUD)

Create Post

- Authenticated users can create posts.
- Title and content are required fields.
- Author and timestamp are automatically stored.

View All Posts

- Displays all posts ordered by most recent.
- Shows title, author, and creation date.
- Clicking a post navigates to full post view.

View Single Post

- Displays full post details.
- Includes content, author, and formatted date.

Edit Post

- Only original author can edit.
- Form is pre-populated with existing data.
- Updates saved to database.

Delete Post

- Only original author can delete.
- Confirmation prompt shown before deletion.

Additional Features (Bonus Implemented)

- User Profile Page displaying posts created by logged-in user.
- Search functionality (search by title/author).
- Basic commenting system on posts.
- Custom 404 Not Found page.
- Loading spinner for API calls.
- Success and error message feedback for user actions.

Security Measures

- Password hashing using bcrypt.
- JWT-based stateless authentication.
- Authorization checks for editing/deleting posts.
- Environment variables for sensitive information (.env).
- CORS enabled for secure frontend-backend communication.

UI/UX Design

The application follows a minimalistic and clean design approach:

- Responsive layout for desktop, tablet, and mobile.
- Dynamic navigation bar based on authentication status.
- Clear and simple forms with validation.
- Post cards for structured display.
- Loading indicators for asynchronous operations.
- Error and success feedback messages.

The focus was functionality and usability over complex styling.

Challenges Faced

1. Implementing JWT authentication and ensuring secure route protection.
2. Managing authentication state persistence across refresh.

3. Ensuring only post authors can edit/delete their posts.
4. Handling asynchronous API calls with proper loading and error states.
5. Structuring backend into controllers, routes, middleware, and schemas properly.

Each challenge helped deepen understanding of full-stack architecture and security principles.

Key Learnings

- Practical implementation of RESTful API design.
- Understanding stateless authentication using JWT.
- Secure password storage using hashing algorithms.
- Full-stack integration between React frontend and Express backend.
- Importance of proper project structure for scalability.
- Handling authentication-based UI rendering.
- Managing MongoDB data models using Mongoose.

Future Improvements

- Implement stronger password validation rules.
- Add role-based authorization (e.g., admin role).
- Implement token blacklisting for enhanced logout security.
- Improve UI responsiveness further for smaller screens.
- Add pagination for posts.
- Deploy application to cloud platform (Render / Vercel).

Conclusion

The Blog Platform successfully fulfills the project requirements by implementing secure authentication, CRUD operations, route protection, and responsive UI design using the MERN stack. The application demonstrates a clear understanding of full-stack development concepts, RESTful API architecture, JWT-based authentication, and MongoDB integration.

This project provided hands-on experience in designing a scalable and secure web application while reinforcing best practices in code structure, security, and maintainability.